Chapter 6. Organizational Structures for SRE

By Brian Kudzia, Christof Leng, Derek Remund, Mahesh Chellappa, Roman Seibel, and Vartika Agarwal
with Carmela Quinito

Site Reliability Engineering is not only a methodology, mindset, or role—fundamentally, it's an organizational construct. Organizational structures that support this construct are important because:

- Site Reliability Engineers do not exist in isolation. To thrive, they need a team around them, and those teams need to be embedded in a broader organizational context to be sustainable and mature.
- SRE is a specialization that is dedicated to building unique expertise in reliable systems. It is not a set of skills that every software engineer is expected to master at the same level as a full-time SRE. However, it does make sense for every software engineer to have a basic understanding of operations—the DevOps philosophy is complementary to SRE in that regard.[1]

The design of an SRE organization needs to adapt to the requirements of the broader organizational context. That said, organizations can pick from and build upon a set of organizational building blocks and best practices for tracking maturity and performance to create their own successful SRE function.

In this chapter, we explore the trade-offs of centralized versus decentralized SRE functions, the design and operation of SRE teams, and how to track the maturity of your SRE practice.

Organizational Structures That Support SRE

An SRE team's practical success is heavily influenced by how lines of control and communication are laid out. Like most discussions of organizational structures, we begin this chapter by referencing Conway's law:[2]

> *Organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations.*

Conway's law recognizes that the social pathways between roles and teams are reflected in the products they produce and support. This scenario is certainly of concern during the initial design and implementation of a product, but it also can produce echoes long into its lifecycle. Standing up an SRE practice is a multiyear endeavor, and your decisions about organizational structure matter: once an approach has been established, it can be difficult to change direction, and you can't expect reality to quickly realign.

Centralization Versus Decentralization

As with nearly any function, you can organize SREs in a centralized or decentralized fashion. Many organizational decisions affect the way SREs deliver services, but deciding between centralization and decentralization is often one of the earliest made and one of the most impactful. Each approach has its own advantages and disadvantages, and inevitably most organizations' SRE implementations fall somewhere on the continuum between centralization and decentralization, to provide a careful balance among those factors. For example, SRE at Google is a centralized organization but leverages a number of mechanisms to also gain advantages akin to a decentralized approach (see "Hybrid Approaches").

In a centralized model, SREs operate from a single-team hierarchy and are responsible for the reliability and operational health of a wide range of services and systems across the enterprise (or the particular subset of the enterprise in which SRE is being rolled out). They all report to the same leaders (e.g., the VP of SRE). In low-trust or high-control organizations (those that demand strict adherence to defined standards and processes with little latitude for uncontrolled innovation), a centralized model tends to be a natural starting point, because the higher-level leadership may not trust the individual suborganizations to align and ultimately deliver. If the impetus and will to create an SRE function are not sufficiently widespread, a central function may be the only viable way to get it off the ground initially. In this early model, it is common for the scope to cover a limited set of services, perhaps as a test or proof of concept.

Decentralized models involve SRE teams operating with a high degree of independence, in alignment with the applications they support. Each team acts within its assigned product organization and may be fully funded and populated by that group. This approach naturally aligns them with their product development partners and can result in improved reliability of each team's specific product. However, it can be difficult or even

scary for low-trust or early-stage organizations to adopt. Leaders who have been charged with reliability improvement in such environments may be uncomfortable giving independent SRE teams the agency to define standards and build tools, and communication channels and organizational structures are often set up to discourage innovation, favoring tight centralized control. In a distributed model, the teams hiring and training SREs have a high degree of autonomy to change or even eliminate the role, while in a centralized model they may feel those occupying the role are inherently outsiders. What's more, trust is bidirectional: low-trust organizations lack the top-down trust that enables decentralized teams to succeed, while those teams lack bottom-up trust in the broader organization and therefore prefer to act independently and retain as much local control as possible.

Perhaps the key advantage of a centralized approach is the ability to concentrate expertise and maintain consistency. While exact technical skill sets may vary from SRE to SRE—often desirably so—mastery of SRE principles such as risk analysis, design thinking, reliability measurement, and blamelessness (see "Blamelessness and blame awareness") is critical to a well-functioning and sustainable SRE practice. Centralized teams can more easily ensure that all SREs are operating from this baseline, and as the state of the art evolves, they can promote consistency in approach across the group. A centralized approach also guards against fragmentation of perspectives; rather than several different teams each with its own ideas about how to build and run reliable systems, a (theoretically) unbiased perspective on reliability concerns can apply across product teams. This unified perspective then makes it easier to organize central efforts such as SRE onboarding and enablement, production health checks, and unified tooling.

Decentralized models seek to accomplish the same goals, but with deeper integration and context and faster feedback loops. Organizationally colocating SREs with the developers and product professionals they support fosters a shared perspective of the service, often reflected in unified project backlogs and an increased sense of ownership. Reduced bottlenecks may improve iteration timelines, and the tight coupling of SREs with application teams can make them strong advocates for the technical and operational needs of the services under their purview. In a centralized model, there can be latency to iteration, both when SREs need to onboard to a service and when services have unique requirements that require special attention.

Context and expertise

Building and running systems at scale is complex and context-dependent work. As the world of production becomes more platform-oriented, SREs may become increasingly focused on the things that make their services unique. Locating SREs more closely to the

services they support makes that context easier to obtain and maintain, as they are immersed day-to-day in the reality of the lifecycles of those systems.

WARNING

SREs that are strongly centralized should take care to avoid a traditional "IT services" mindset, where interactions are transactional and the focus becomes simply maintaining a steady state rather than advancing the quality of the product.

Simultaneously, SREs must ensure they are increasing their impact by generalizing the solutions they create for their services, working across their community of practice so that improvements to areas such as observability, production readiness, and toil management benefit the full array of products. Ultimately, such solutions may become part of a standard production platform.

The distribution of expertise among individuals can weigh heavily toward centralization in nascent SRE organizations. Simple numbers in terms of personnel and their skills may force the centralization of some reliability functions, but this can pay dividends as well. SREs need each other, and the more colocated they are organizationally, the less friction there tends to be for knowledge transfer, individual upskilling, and cross-pollination of approaches to problem solving, risk mitigation, and incident response. As the organization scales, the centralized function can continue to optimize by rebalancing teams as required while also seeding new teams with experienced SREs.

A strong social culture of psychological safety and blamelessness (see "Psychological safety") coupled with an engineering culture dedicated to improving systems and unlocking velocity can counteract some of the negative pressures of centralization (discomfort with loss of local control, resistance to outside mandates, and so forth), and that culture may be easier to establish and maintain with a more centralized organization. Culture (see Chapter 4) is ultimately what teams fall back on during tough situations, and centralized SRE groups often have a depth of support that makes deviation from those tenets less likely and more quickly corrected.

Ownership

In the best-case scenario, the high-context focus of a decentralized approach leads to a strong sense of ownership in SREs. SREs defend the reliability targets of their products, which can engender a sense of camaraderie with the product development team. This is

not to say that centralized SREs don't care about their services, but ownership is more natural when the scope is narrower and the partnership with developers is closer.

When capturing the ownership gains of decentralization, be careful not to return to an old-school "us-versus-them" mentality. While SREs are dedicated to their individual services, those services are likely intertwined with others, and the entire service chain and product portfolio are important to the business. Cross-team events and exercises, unified onboarding and training, and a well-defined SRE role can all help keep the teams happy (Chapter 5 covers this in more detail).

Career path

As technical professionals move upward in their careers as individual contributors, they often tend to become more specialized. This tendency can lead to rewards initially, but ultimately may result in pigeonholing and stagnation of development. SREs are as prone to this stagnation as any other role, and clarity of career path can encourage them to focus more on doing great engineering than on managing their growth. Centralized teams make it easier to establish consistent pathways for growth, although if individuals do stagnate, fewer alternatives exist without leaving the company (see the discussion of internal mobility in "Collaboration"). Neither approach has a definitive advantage, but pay attention to aligning day-to-day work with the things that engineers find satisfying and growth-promoting.

Economies of scale

Any transformation effort inevitably must address questions of cost; indeed, many begin for the purpose of controlling cost of service. A naive examination of SRE may consider it a high-cost approach to providing a service, as it uses very skilled labor. But once economies of scale and improvements to reliability are taken into account, it can be a very cost-effective approach. Presuming you've chosen an SRE model, decisions about implementation then have further impacts on the economics of reliability.

Engineering work is, of course, one of the primary costs of providing reliability. Centralized teams can more easily avoid duplication of effort by more directly controlling tool proliferation and process consistency, but they must take care not to stifle innovation or create organizational overhead. Often, the best ideas for new approaches come from individual services meeting their own needs, the results of which can then be generalized to other teams and services.

Some personnel efficiencies may also exist with centralized groups, as roles such as program management can be shared, and fungibility of SREs may make it easier to shift effort between services as their needs change. However, the most efficient model on paper is not always the most efficient in practice, as an overfocus on efficiencies can lead to operational overload (see "Situational Awareness") and lower reliability across products. Toil management (see "Effective Strategies for Addressing Toil") becomes ever more important in these cases.

Hybrid Approaches

Even when a set point is carefully chosen along the continuum between centralized and decentralized, drift will inevitably occur. This drift is desirable to a point, as smart teams will adjust their ways of working to be more efficient and productive. However, you should avoid extreme adjustments, which can result in "shadow SRE," along the lines of ad hoc IT functions that often pop up in dysfunctional organizations.[3]

Several strategies exist for establishing a starting point as a happy middle ground to minimize drift. Some examples include:

Maintaining a purposeful community of practice

A strong community of practice (CoP) among practitioner SREs can provide many of the benefits of strong centralization, but it does require advocacy and resources to maintain. A CoP can provide continuing education, internal events, minimum practice standards, and similar coordinating efforts even in more decentralized settings. It's usually led by a selection of practitioners and leaders, with a mandate to involve SREs and related roles at all levels. Contribution to the CoP should be encouraged as impactful work.

Establishing clear engagement models

Centralized SRE groups can provide more effective and higher-context support to their associated products through engagement models that define clear inputs, outputs, time frames, and other expectations (see Chapter 7). Standardizing these models shortens the adoption cycle and leads to higher-quality delivery, as SREs and product teams have less negotiation to perform and can focus more on context and quality.

Building a strong SRE program management function

Technical program managers (TPMs) are important enough that Google has an SRE-specific TPM role. Much of SRE's effectiveness depends on the feedback

loops among SREs, product engineers, and product and business stakeholders, and maintaining the context and consistency of those interactions is core work for an SRE TPM. TPMs keep the machine turning and can iron out inefficiencies generated by your chosen level of (de)centralization.

An example hybrid model might be established as follows:

- A strong senior leader heads a centralized SRE group.
- The centralized group provides a variety of functions, including:
  - Time-limited reliability engagements of varying depth with application/ product teams, such as reliability review, SLO construction, or postmortem facilitation and follow-up
  - Standardization of production readiness standards
  - Engineering muscle to build tooling and generalize solutions innovated by application teams
  - Leadership of the SRE community of practice, including ongoing enablement, as described previously
- Products with a combination of sufficient size, complexity, uniqueness, and/or importance have dedicated, permanent SRE teams aligned with their structures.
- SREs on dedicated teams are onboarded through the central team, and SRE culture is maintained through the CoP, standardized practices and role performance guidelines, and continuing enablement and engineering exercises.

Making SRE Transformation Work

Much like residents of many cities seem to think they have the most variable weather, or the worst drivers, quite a few engineers will complain that their teams are reorganized more than any other. SRE-driven companies are not immune to this tendency. Reorgs come with downsides in opposition to their intended upsides, such as:

- Trust and morale deficits, and even cultural damage
- Context losses with supported product teams
- Destruction of the webs of common practice among teams, particularly if less formalized
- Organizational memory that resists or routes around change
- Extra load during the trough of the J-curve

Given the risks associated with change, core SRE strategies such as gradual rollout (in this case, of organization change) are critical. Whichever direction you are moving along the centralization continuum, consider identifying first-mover services or products that

are critical enough to require SRE support and representative of typical strategy and architecture. As these teams adopt the new approach, focus on tight iterations and frequent feedback. The use of metrics, whether related to incidents, focused on software delivery (such as DORA), or based on sentiment analysis, can support the continued rollout of your changes.

In the interest of showing how organizations other than Google have implemented successful organizational structures for their SRE teams, the following case study provides a look at how Spotify approaches SRE.

Spotify's Distributed Reliability Engagement Model

By Laura "Morgy" Morgan and Oskar Arvidsson with John Stäck and Niklas Gustavsson

Spotify provides a global, latency-sensitive consumer platform with a complex service ecosystem and continuously evolving client surfaces. In this environment, reliability is a first-order product concern alongside development velocity and cost efficiency. Spotify's engagement model illustrates a stable alternative to a centralized SRE organization: *operational ownership remains primarily with the product and infrastructure teams that build and maintain systems*, while the organization invests in shared capabilities that keep that ownership sustainable at scale.

This case study focuses on Spotify's reliability engagement model. It describes how reliability work is initiated, supported, and scaled when responsibility stays distributed across teams, and what enabling mechanisms are required to make that approach durable.

Reliability Ownership Without a Centralized SRE Organization

In many organizations, "SRE engagement" is synonymous with a central SRE team taking on a defined set of services (often including primary on-call). Spotify's model is intentionally different and rooted in a simple operational principle: *the teams that own services also operate them*.

In practice, this typically means that they manage their own on-call rotation, handbooks, alerts, and monitoring. They are expected to properly mitigate incidents as they happen as well as follow up after incidents. Historically, this was described as *Ops in Squads*,[4] where a squad (a small, cross-functional development team) operates the services it ships.

Spotify keeps accountability close to where systems are built:

- Product and infrastructure teams own the reliability of the services they build and operate.
- Reliability work is not automatically "pulled" into a central group; instead, the platform engineering team provides capabilities to teams through shared standards and tools so that reliability becomes a built-in part of the development cycle.
- A small, central function supports cross-team incidents in cases where coordination, rather than hands-on troubleshooting or mitigation, becomes the limiting factor. At Spotify this function is called IMOC; we'll go into more detail on it in the next section.

This structure does not imply that every engineer is a reliability specialist or that teams operate in isolation. Rather, it assumes that reliability practices become part of normal engineering work, supported by organization-wide mechanisms that reduce variance and make "doing the right thing" the default.

Coupling development and operations with ownership tightens the feedback loop between engineering choices and operational outcomes. When teams are on-call for what they build, operational costs are immediate and visible: noisy alerts, slow mitigations, or unclear failure modes translate directly into time spent by the people closest to the code. That feedback loop shapes both prioritization and design.

In practice, this model reinforces several behaviors:

Shorter feedback cycles

Reliability issues surface quickly and can be addressed through normal development work rather than risking being deferred indefinitely.

Aligned incentives

Teams have incentives to invest early in observability, safe deployment mechanisms, and clearer operational interfaces.

Contextual trade-offs

Decisions about availability, latency, degradation, and cost are made in direct product context rather than in a centralized SRE team with less domain knowledge.

This approach scales well for day-to-day operations and for incidents where a single owning team (or a small set of teams) has clear responsibility. However, at a larger scale,

a distributed model needs additional structure for incidents that span boundaries or lack obvious ownership.

Incident Management as a Shared Capability

Spotify's incident response model resolves most incidents efficiently because the owning team typically has the context, access, and incentives needed to diagnose and mitigate issues. In the common case, adding a coordinator would slow response rather than help it.

At Spotify's scale, however, a smaller but critically important class of incidents exposes the limits of distributed ownership alone: incidents where ownership is ambiguous, where multiple teams must coordinate under time pressure, or where decisions cut across organizational boundaries. In these cases, the primary bottleneck is often *coordination, clarity, and decision making* rather than technical execution.

To address this gap, Spotify maintains an *Incident Manager On-Call* (IMOC) rotation. This is a company wide on-call rotation scheduled to follow the sun and staffed by senior engineers with strong incident response experience. IMOC's role is intentionally narrow in scope:

- IMOC does *not* own services.
- IMOC does *not* routinely participate in incident response as a responder.
- IMOC does *not* take over diagnosis, mitigation, and remediation when engaged.

IMOC is engaged when coordination becomes the limiting factor, while owning teams retain responsibility for diagnosis, mitigation, and remediation. Any team can escalate an incident to IMOC, typically when one or more of the following conditions apply:

- The incident spans multiple teams, ownership is unclear, or the incident is evolving across boundaries.
- The impact or duration requires sustained delegation, broader visibility, or escalation beyond the immediate responders.
- Cross-cutting decisions or escalations are required (for example, risk trade-offs, rerouting traffic, or engaging external parties).

From an engagement model perspective, IMOC functions as a *scalable capability*: it can be applied broadly without deep service-specific knowledge, and it is engaged dynamically based on incident characteristics rather than fixed service ownership. Its purpose is to reduce cognitive load on responders by coordinating communication, clarifying own-

ership, and enabling timely decisions—so teams with domain expertise can stay focused on mitigation.

Scaling the Model

To sustain its core model of distributed ownership amid rapid growth, Spotify has prioritized reducing operational variance. As the ecosystem's diversity grows, scaling distributed operations becomes increasingly nonlinear: coordination overhead and reliability work rise faster than the number of engineers, especially when each team maintains a unique stack and set of practices. A fundamental lesson learned is that scaling distributed operations is much easier when infrastructure, service design patterns (frameworks and libraries), and tooling are standardized. With strong standards, operational expectations and expertise transfer across the fleet. It's as much about reducing variance as it is about adding staff.

Spotify has made significant investments in shared platforms and standardization to make reliability improvements repeatable and systemic. It's able to do this by investing in central bodies that drive specific efforts around reliability and operations.

For example, the Platform Engineering organization focuses on:

Standardized stacks

Providing opinionated service frameworks, system design, and production service standards

Centralized platforms

Implementing company-wide platforms and practices for deployment, observability, and incident response

Automation

Utilizing platform teams to enforce baseline practices and enable large-scale automation

Observability and metrics are part of that standardization. The platform provides a common observability toolchain and metrics foundation that teams get by default when they adopt the service framework: baseline dashboards, standard alerts, and consistent service-level signals come bundled via standard monitoring and alerting configurations. Teams can extend this with domain- and feature-specific indicators, while the platform

provides rollups from services to teams so failures, blast radius, and impact can be understood consistently across the company.

These initiatives lighten the cognitive load on individual teams and ensure a more uniform level of reliability by default. They also foster a *fleet-first operational mindset*, where services are managed as a continuously evolving fleet rather than disparate systems. This approach supports frequent, automated changes, keeping the systems healthier and reducing the risks associated with less frequent, major migrations.

While standardization has been an ongoing effort, it has accelerated recently. By utilizing mechanisms to ensure production software follows Spotify standards and moves both client and backend fleets to monorepos, Spotify is unlocking significant opportunities for central, platformed management. Crucially, this positions the company to better leverage automation and AI to offload tedious, toilsome duties that currently detract from core product and platform development.

To sustain these initiatives over time, Spotify provides governance, guidance, and policy around these standards using bodies and functions that, while central, represent the wider engineering community. The Reliability Advisory Board, comprising senior engineering representatives from across Spotify, partners closely with Platform Engineering to ensure reliability standards and expectations are both well supported and consistently met. Complementing this, the Backend Advisory Board provides similar stewardship earlier in the lifecycle, guiding service design patterns and backend standards that underpin reliable operation at scale.

Concurrently, operational ownership has shifted toward *Ops in Areas*, where an Area is typically a set of closely related squads (at Spotify, we call these Product Areas). An Area aligns incident response and coordination, which is better suited for larger systems where reliability concerns span multiple services and dependencies. As stacks become more homogeneous, operational work becomes easier to share, reinforcing the incentive for teams to converge further on common practices and platforms.

Trade-offs and Lessons

Centralized SRE organizations are a proven and effective engagement model. They can concentrate expertise, drive deep reliability improvements, and enforce strong production standards. Many organizations benefit from that approach, especially when tight control and consistency are paramount.

Spotify's model represents a slightly different equilibrium: both accountability and operational responsibility remain *within the product team*, while the organization invests centrally to enable that accountability at scale through standardization, shared platforms, and targeted incident coordination.

There will always be trade-offs with either path, and for us those are sometimes very real:

- Distributed ownership of reliability requires sustained investment in production standards, opinionated service patterns, and fleet-managed upgrades. Without standardization, autonomy can devolve into inconsistency and operational overload.
- Incident facilitation must remain carefully scoped to avoid turning escalation into an implicit handoff of accountability—for example, IMOC should not become the default escalation path for routine incidents. By maintaining a healthy, blameless culture around incidents, we are better able to engage with that scoping.
- Reliability expertise must be grown and shared through platforms, practices, and learning mechanisms, rather than assumed to live in a single specialized org.
- Accountability at scale depends on platform-provided observability defaults and higher-level insights, so teams can quickly connect service failures to user impact, and the organization can track reliability outcomes consistently and drive clear follow-up across the company.

Specialized product expertise allows us to solve complex challenges. To fill the gap in deep reliability engineering expertise, we foster a continuous learning culture by providing courses, detailed documentation, and structured training from day one. This investment in learning, technology, and infrastructure drives our operational excellence. We then leverage our Reliability Advisory Board to institutionalize and govern organization-wide best practices, setting standards and monitoring adherence. This ensures a unified, high-standard reliability approach across all operations.

The broader takeaway is not that one engagement model is universally superior. The more durable lesson is that engagement models succeed when *ownership boundaries, interfaces, and shared capabilities* align with the organization's scale and product needs. Reliability emerges less from org charts and more from how work is initiated, how decisions are made, and how the system is made operable by default.

Team Design

The SRE team is the nucleus of the SRE organization. A singleton SRE is not sustainable. They are a single point of failure. They have no one to learn from or to exchange ideas with. They can't take on bigger responsibilities, like multiperson projects or staffing an on-call rotation. If you don't have the headcount to staff a small team, it is often better to establish a DevOps mindset within product development and make production expertise a shared responsibility.

Healthy teams with a purposeful design are the foundation of SRE work. To achieve this, every SRE team needs a clear scope and mission, and the necessary resources to fulfill this mission (see "Scoping SRE Engagements"). At Google, every SRE team is required to have a charter document approved by its leadership to ensure this clarity.

The charter (see Appendix B) defines the mission statement of the team, along with its scope in terms of technology environment, the product development partners it works with, its offerings, and the services it's working on. The scope should not be a collection of random items but should form an interconnected whole. Because SREs are outside the responsibility of any individual product development team, they can be very effective at building an end-to-end view of neighboring services and shared technologies, as well as identifying and addressing risks and opportunities that might otherwise be overlooked.

The charter also defines what success looks like for the team. These success criteria should be defined as measurable outcomes that have a substantial positive impact on the users and the business. As discussed in "An Outcome-Focused Approach", outcomes should have a timeline attached to them. Teams are expected to regularly review and update their charters over time. See Appendix C for a hypothetical charter example.

The general rules for building successful teams apply to SRE teams as well. The research findings on team success and foundational qualities of successful teams that Google has published on Project Aristotle can be very helpful to build a healthy SRE organization. This research stresses the importance of psychological safety, which is especially relevant to SRE.

Team Roles

There are a wide array of skill sets in SRE, ranging from systems engineering through software engineering and into organization and management. This diverse set of skills and backgrounds enables SRE teams to adapt to a wide range of technically and organizationally complex and challenging environments.

Formally, an SRE team at Google includes the roles of manager, tech lead (TL), and technical program manager. The remaining team members are usually called individual contributors (ICs). Not every team staffs all three of those roles; some are more successful with only two of the three. Some people operate best when the roles have well-defined responsibilities, the major benefit of this being that they can make in-scope decisions quickly and safely. Others operate best in a more fluid environment, with shifting responsibilities depending on dynamic negotiation. It's crucial to pick a level of formality that lets the individuals on the team thrive and fits the culture of the wider organization. Depending on the team and organization, that may be dictated from the top or self-organized by the team.

Let's consider each role in more detail:

- The *manager* is both a technical and organizational leader. They set the team's strategy, encompassing development, culture, execution, performance management, stakeholder management, and alignment with an organization's business objectives. The manager also handles staffing and organizes work assignments for team members.
- The *tech lead* is deeply knowledgeable about the domain area and defines the technical vision. They are responsible for the technical direction of the team and the health of the services in scope. They coach and encourage good technical practices within the team and with product development partners, collaborating closely with those partners and the broader SRE community.
- The *technical program manager* acts as a force multiplier and improves the quality and velocity of the team's work. They handle program/project management, planning, process design, stakeholder management, and communication. The TPM often works at cross-functional boundaries and enables fast decision making and forward momentum. They are responsible for efficient delivery of programs that can span multiple organizations and locations. To ensure that the planning is founded in engineering reality, they should possess the technical judgment of an engineer.

All three roles work together to prioritize work and define the quarterly plan in team objectives and key results (OKRs) and operational work. The other engineers on the team are also encouraged and, depending on their seniority, expected to provide input to planning and team direction.

On-Call Considerations

An SRE team must balance engineering projects and operational responsibilities. At Google, the guideline is that an SRE team's time must be devoted to at least 50% engi-

neering work such as coding, analysis, and design, and at most 50% operational work such as on-call duties, tickets, and maintenance. No more than 25% of an SRE's time should be spent on on-call duties. These figures are thresholds, not targets. In general, we aim for a substantially lower fraction of operational work. The actual operational workload of a team depends on many factors, including maturity of the services, breadth of scope, and user growth rate.

In our experience, the ops work thresholds can be best maintained when the SRE team has a certain minimum size. Many SRE teams at Google are split between two sites, separated by a time zone difference of six to nine hours. This setup enables a "follow-the-sun" on-call model, where SREs at each site are on-call during their day and evening, but not at night. Larger time zone differences make team collaboration even more difficult. In this two-site setup, a team should have at least six engineers per site. This setup works out to approximately two to three on-call weeks per quarter for each engineer (15–25% of their time), leaving enough uninterrupted time for project work while not risking them getting out of practice on incident response. In a single-site setup, where the engineers are also on-call at night, the team should have a minimum size of at least eight on-callers. Teams can (and often should) grow beyond these minimum sizes, but when a team reaches twice the minimum, it typically makes sense to split it into two teams to minimize the cognitive load on the engineers. Growing and splitting teams is much less disruptive than creating new teams from scratch.

Ideally, the operational work can be fully handled by the current on-caller, representing about 15% of the team's capacity (e.g., 1/8 or 2/12) based on these minimum team sizes —well below the 50% threshold.

It is possible to have SRE teams with less than eight engineers, but in such scenarios a more conscious and disciplined approach to ops workload monitoring is required to ensure that the team stays below the 50% ops threshold that ensures the SREs have sufficient time for improvements through engineering work.

Collaboration

SRE is an inherently collaborative role. It spans technical domains and team boundaries, both across SRE teams and across the product development organizations. To be successful, SREs must master the skills of leading without authority and building consensus in loosely coupled groups—often for ideas and proposals that may look radical and disruptive.

Aside from organizational boundaries, many SRE organizations and their product development partners are geographically dispersed. Collaborating effectively across time zones requires strong written communication skills and opportunities to travel. In-person interactions are deferrable but ultimately necessary for a high-quality relationship, to pay off the "relationship debt" (analogous to technical debt) that accrues over time. Modern online collaboration and videoconferencing tools can help keep the relationships alive in between, but we recommend meeting your most important partners and colleagues in person at least once per year. Otherwise, even if you're a great writer, you risk becoming just an email address.

Product Development

At Google, an SRE team always has at least one product development partner. If a service is not actively developed, the team has limited opportunity for impact, and it is unlikely to be the best investment of SRE resources.[5] An SRE team is a software engineering team responsible for minimizing the need for ops, not taking over the role of the ops team. In practice, most SRE teams at Google partner with multiple—sometimes more than 10—product development teams. For more on interactions with product development, see Chapter 7. While this setup is organizationally challenging, it enables Google SREs to build a broad view of the technical ecosystem, transfer ideas across teams, and optimize system health at a global level.

On the operational side, Google SRE encourages shared product development and SRE on-call rotations. For example, product development teams can be on-call during their business hours with SRE as an escalation path, and SRE can be primary during off-hours with product development teams as the escalation path. This setup helps product development teams to get direct exposure to production, which enables them to design and build robust and manageable systems while still benefiting from SRE's operational expertise and support during critical incidents. In cases where the primary on-call rotation is exclusively staffed with SREs, product development teams are generally expected to maintain a 24×7 on-call escalation path through a secondary rotation. While SREs can handle most incidents on their own, sometimes engineers with detailed knowledge about the latest code changes are crucial to quickly identify nonobvious triggers for an outage and their mitigations.

On the project side, the SRE and product development teams at Google should engage in shared planning and prioritization. Ideally, this work has strong buy-in from the senior leadership of the product development partner organization and is closely tied to business outcomes. When an SRE team's business value is not well understood, it risks becoming irrelevant to both the business and its development partners. SRE and product

development teams should maintain a shared backlog and roadmap that both sides contribute to and execute. Generally, the team that is best positioned to deliver a particular piece of work efficiently and with high quality should own it.

Close collaboration between engineers from product development teams and SRE helps build visibility, mutual understanding, and trust. The teams should meet regularly to keep information flowing in both directions. The most fundamental of these meetings is the production meeting. Other shared activities that focus on a longer-term perspective are quarterly business reviews, shared OKR planning, and roadmap planning.

PRODUCTION MEETINGS

SRE teams at Google have a weekly meeting with the whole team, typically called an ops review or production meeting, to discuss:

- Significant incidents
- Upcoming changes
- Relevant project work
- SLI performance
- Bug triage status
- Bug SLO compliance

This meeting ensures visibility into all production-relevant topics across all services for every member of the team. Production meetings are the cornerstone of an SRE team's work as a whole. They are not about individuals giving project updates; they are service-oriented and capture the health of production.

When an SRE team works with only one or a small number of product development partners, inviting them (or at least representatives, such as their TLs) to the production meeting is a good practice. However, when a team has many product development partners, this scenario quickly becomes unwieldy. In this case, it should run separate production syncs with each of its product development partners in addition to its own production meeting. These syncs can take place weekly or fortnightly and do not require the whole team; delegates from the SRE and product development teams can relay the communication to the other team members.

Production meetings tend to be highly structured rituals with a prepared template and agenda. This helps maintain discipline and avoids wasting time or overlooking relevant issues. The typical agenda consists of the following topics:

Outages and pages

Any incident that resulted in an on-call page should be discussed. Was the monitoring working as intended? Were the root causes identified and the problem fixed in such a way that the probability of recurrence is low? Noteworthy outages deserve a postmortem.

Postmortems

Have all relevant outages received a postmortem? Was it reviewed and action items executed? Many SRE organizations run separate, additional postmortem reviews with their product development partners to increase their visibility across team boundaries.

SLOs and service health

Are the service health numbers where they need to be? Are there worrying trends that require the team to take action?

Non-paging alerts and tickets

Are you staying on top of alerts and tickets, and are any changes needed? These operational tasks interrupt work and can often take up a significant amount of SRE time. Keep track of the workload and ensure that no backlog is building up. The production meeting is a great opportunity to identify alerts that should be eliminated because they are not actionable or relevant, or that should be promoted to paging events because they turn out to be too critical to be left in the ticket queue. If a relevant problem was detected manually, updating the monitoring to detect it automatically in the future may make sense.

Upcoming and in-flight production changes

Is the product development team planning any launches or service changes that may impact production? Are large increases or decreases in load expected? Is SRE working on any infrastructure changes that the on-callers should be aware of? The goal is not to stop changes but to raise awareness and ensure that they get executed safely. The visibility helps derisk potentially conflicting changes and enable faster fixes if a change has an unexpected effect or outcome.

While the structure outlined here helps facilitate collaboration during production meetings, the work doesn't end there—there needs to be follow-up action. It's best practice to write down action items and review their resolution in subsequent production meetings. See Appendix D for an example of the minutes from a production meeting.

SRE-Internal Collaboration

SRE at Google has a healthy culture of small teams empowered to solve problems quickly without getting bogged down by significant process or administrative overhead. And we want to keep it that way. The downside, however, is that this setup has sometimes resulted in a proliferation of overlapping and redundant solutions, leading to wasted time and effort. There is also an opportunity cost in failing to build a high-quality ecosystem overall. Training and ramp-up can be difficult, and the lack of standards limits the transferability of knowledge within SRE. To address these problems, SREs are encouraged to "look left and right" before building custom solutions.

An SRE team works across many services and has good visibility into their common production needs. While many problems are highly service-specific, more often than not the same solution applies to a class of problems that many services share. This is even more true when you look across team boundaries. So, before an SRE team decides to build custom automation, it needs to investigate whether that problem is truly service-specific. If not, the next step is to research existing tooling that others have built and see if it can be applied to the problem. This step may involve contributing to the existing solution to make it reusable in more scenarios. In cases where no solution exists yet, the SRE team should keep reusability in mind. Sometimes these initially local solutions grow into an infrastructure product that becomes widely used across the company.

When a team comes up with a novel solution, it should communicate that solution through the appropriate channels. Tech talks are an effective way to do so. Since an engineering site typically hosts SRE teams from multiple product areas, local tech talk series can create visibility far beyond the immediate organization. These talks can cover software tools, new methodologies, experience reports, processes, and collaboration models. Solutions that have shown value repeatedly in a broad set of scenarios can be canonicalized into training modules to educate new SREs or production-interested developers.

More than ideas migrate across team boundaries. Personal mobility is another important aspect of the SRE culture. Engineers moving freely from one SRE team to another, or from SRE to product development or vice versa, can disseminate important institutional knowledge and pollinate a widespread culture of reliability.

Google SRE supports this mobility and culture exchange with a number of programs, such as:

Mission Control

: Developers can join SRE from other teams for a six-month rotation.

SRE Exchange

: SREs can join another SRE team for a week, learn how they work, and report what they learned back to their home team.

N+1 mentors

: Mentoring for new SREs across team boundaries helps them gain a broader understanding of SRE and start building their networks inside the company.

Managers cannot stop their engineers from transferring to another team—this setup is intentional. Engineers can transfer out of teams that are not a fit for them.

Maturity Matrices

The amount and nature of SRE involvement depends heavily on organizational maturity across several dimensions, including:

Operational processes

: Activities and metrics used to maintain production health

Risk management

: Evaluation, identification, and mitigation of production risks

Productivity

: Impact of checks and balances on feature launches

Velocity

: Rate at which new features and capabilities are delivered to production

System complexity

: System architecture, paired with the dependency graph of a product or service

Product lifecycle

: Stage in the lifecycle, from incubation to decline

Teams

: Operating model, values, and team ethos

Leadership

: Commitment of leadership to engage beyond siloed ownership[6]

The intention of the maturity matrix is not to provide a score or act as a checklist but to determine where an organization's maturity lies on a continuum so that SRE involvement can be dialed in and out accordingly. Different services may be at different maturity levels at any given time.

SREs have a unique perspective because they are exposed to a broader view than a product development organization. That unique vantage point often enables SRE teams to foresee reliability issues and risks earlier and more clearly. Since SREs are often scarce and in high demand, choosing the right SRE engagement model based on the maturity matrix yields the maximum returns. The engagement can range from foundational services provided by SRE (with or without shared maintenance) to full production support owned by SRE.

An appropriate engagement model ensures a healthy SRE organization and reduces reliability risk for the services and systems it supports. The health of the SRE organization can be understood across three dimensions:

Organizational health

: Purpose, staffing, and operating model

Operational health

: Balance of engineering and operational work, and opportunities for automation

System health

: Unresolved risks and reliability gaps in the systems

These aspects reflect the level of risk to the production environment. To reduce this risk, it is critical to accurately assess organizational maturity and apply the appropriate engagement model. It is also important to develop a strategy for progressing toward the maturity level that yields the greatest return on investment for SRE engagement. The different levels, or phases of organizational reliability, can be arranged along a continuum, as follows:

- Absent
- Reactive
- Proactive
- Strategic
- Visionary

These phases describe the organization's overall mindset. Each one is associated with a certain set of attributes. An organization may operate at different levels of maturity across different attributes, depending on its focus and strengths.

To better understand the continuum, let's take a closer look at each of the phases.

Absent

In this phase, product functionality is the main driver for the organization, with reliability being secondary. The following characteristics denote the absent phase:

- Feature launch with high velocity is the key organizational metric and the focus for incentives.
- A majority of issues are found by users or testers. The organization is not aware of long-term reliability and availability risks.
- The organization recognizes those who work on feature or product launches through bonuses, public callouts, and promotions, but it does not reward efforts to reduce technical debt or improve production quality.

Having SRE involvement early in the lifecycle, even during the architecture phase (see Chapter 7), could avoid reliability and scalability being treated as an afterthought and prevent an expensive rearchitecture or redesign in the future. Even a small investment in SRE at this stage can help ensure systems are built with the reliability, scalability, and operational capabilities needed to meet user expectations. As product adoption expands, it's important to consider an increase in SRE engagement to maintain the production system while minimizing operational load and risk.

SREs will typically prioritize their engagement with services that are business-critical or have high user impact. When a service enters a declining phase of its lifecycle, user adoption and the rate of new feature development typically decreases. This presents an opportunity to reduce SRE engagement and reallocate capacity to other priorities that are business-critical—a transition that is both expected and efficient.

In other words, this phase could be an intentional choice for the organization in these scenarios:

- The service is still in the incubation phase and not yet available to the public or customers.
- The service is in a declining phase, with maintenance work fully automated and feature velocity and churn at a minimum.

Reactive

In the reactive phase, the SRE organization is focused on fixing recent outages with sporadic follow-through. Longer-term investments in fixing system issues are rare. Product

development and SRE teams work independently. This phase is characterized by the following:

- SREs write postmortems for outages and create action items for tactical fixes.
- Some metrics exist, but they are not comprehensive or monitored over time.
- Heroic efforts by a few individuals or teams maintain reasonable availability.
- Developer productivity is throttled due to temporary shifts in priority due to outages.
- Feature development may be frozen for short periods of time.

This phase is appropriate for products or projects in pre-launch or in a stable long-term maintenance phase. Due to the reactive mode of maintenance, SREs and product development teams have an increased operational load that can deter them from delivering engineering work. Since the SRE and product development teams operate in silos, the organizational health and accountability of individual teams are also impacted. Unmanaged risks and unknown deficiencies affect the health of systems. Identifying this phase and shifting the organization's maturity into the next phase is critical to restoring the health of the production system.

Proactive

SRE has a well-defined engagement with the product development organization in this phase. The teams work together seamlessly, with joint ownership of production health. Regular organizational processes identify and address potential risks. The following characteristics denote this phase:

- The product development team and SRE team regularly review and prioritize risks, and the two functions work together as one unit.
- Teams proactively manage dependencies and review their reliability metrics, such as SLOs.
- Product development and SRE teams jointly own service reliability and may be on the same rotation for production support.
- The teams assess new designs for known risks and failure modes early on. Graceful degradation is a basic requirement (except where degradation cannot be applied, such as in policy-enforcing systems, where you can't let some policies just fail open on load).
- Both SRE and product development organizations prioritize headcount and investment.
- The teams proactively look at the manual toil in the operational load and work together to automate and reduce toil.

Ideally, most products should be at least at this level—particularly if they have a large blast radius or are critical to the business. Once an organization's maturity reaches this phase, it should invest in moving to the strategic phase. Taking this step will improve organizational health and morale, enabling a shift from tactical work toward more engineering-focused and innovative work.

Strategic

Organizations at this phase manage categories of risk via systemic changes to architectures, products, and processes. SRE engagement is similar to that in the proactive phase, but SREs engage early in product development to instill reliability principles right from the architecture and design stage. This phase is characterized by the following:

- Mature processes are inherent and ingrained in how the organization designs, operates, and develops software.
- Complexity is addressed holistically through product architecture. Dependencies are constantly reduced or improved.
- New architectures and design rewrites are prioritized to improve the reliability and/or scalability of existing systems.
- SRE and product development teams can sustain reliability and developer velocity simultaneously.
- Both teams widely celebrate quality and stability milestones.

The strategic phase is appropriate for services and products that require very high availability to meet business-critical needs. SRE engagement is strategic in this phase, rather than tactical, as it is in the proactive phase. As mentioned previously, this setup increases organizational health and improves morale within the SRE team through the satisfaction of delivering meaningful engineering work. The production systems have minimal or well-controlled known risks. When toil is significantly reduced, SRE involvement can be scaled down and redirected toward engagements that provide a better return on investment.

Visionary

An organization at this phase has reached the highest level of reliability and is able to drive best practices based on its experiences and broader reliability efforts within and outside the company—for example, by writing papers and sharing knowledge. At this point, SRE can shift its focus and move to other areas that need more attention. The following characteristics denote this phase:

- Production health exists broadly across the organization. Production knowledge exists broadly across all engineers and teams.
- Systems are self-healing and can automatically recover from failures and maintain operations without human intervention.
- Architectural improvements positively impact productivity and release velocity due to reduction of maintenance work/toil.

Very few services or products are at this level, and those that are lead the industry.

Driving the maturity metrics within your organization incurs significant costs. Reaching the strategic level can take four or more years and requires significant investment across the business. Without ongoing focus and sustained investment, it is very easy for an organization to regress.

It is important to note that the optimal maturity level for an organization can vary over time, for different reasons. While most organizations will progress along the continuum as they mature, regression is possible (e.g., from proactive to reactive) if rigor is not maintained. Furthermore, the model is not strictly linear: there are scenarios where product usage and the blast radius are contained (e.g., when the product is nearing the end of its life) and where it may be reasonable for maturity to shift between levels. Maturity metrics are neither a prescriptive list of tasks nor a set of mandated principles. The key point is that these trade-offs are intentional, based on an organization's context and journey.

Scorecards

Scorecards are a popular mechanism to track an organization's key performance indicators (KPIs) and progress within a maturity matrix in the form of dashboards. They can be useful when used with care but can backfire when used naively. The intention is to condense strategic production health into quantifiable metrics that are actionable. While they are neither categorically useful nor not useful, they're often institutionally mandated.

W. Edwards Deming wrote, "It is wrong to suppose that if you can't measure it, you can't manage it—a costly myth."[7] Yet, measuring can make some things easier to manage. And there's also the risk that if you don't measure something, it tends to become worse.

So, while measurement isn't a prerequisite for management, a set of KPIs can help simplify a complex problem. They build alignment within the group on what is important

and how to measure success. Agreed-upon objective metrics can reduce interpersonal conflict driven by subjective perception. If your SLOs are green, SRE and product development should not have to debate whether the service is healthy or not.

Why Do They Fall Short?

However, scorecards are not without risks. Good KPIs can be very hard to define—especially because modern production software systems are inherently complex and are often multiple levels removed from direct business outcomes like revenue or customer retention.

Almost all KPIs are proxy metrics. They don't measure the elusive business performance directly but rather something that is more easily measurable. In other words, KPIs correlate with business outcomes but are not the same; 100% alignment between those two does not exist.

This mismatch can have negative consequences in both directions. First, leadership may "weaponize" the metrics to put pressure on the engineering teams. Teams that don't have sufficiently good numbers may face consequences. Leaders may not be interested in understanding the situation the team is in, or why it was not in a position that allowed it to make the desired progress. "Managing by numbers" without empathy or deeper understanding undermines psychological safety and spreads fear instead of nurturing change. As Jeff Bezos once put it: "When the data and the anecdotes disagree, the anecdotes are usually right."[8]

Second, teams that are not committed to the spirit of the scorecards may find opportunities to game the metrics. When you measure your CI/CD performance via deployment frequency, as the widely adopted DORA metrics suggest, a team may choose to deploy the same binary every hour instead of releasing new binaries more often. That doesn't actually improve CI/CD performance, but it looks great on the scorecard.

Goodhart's law, which states that "when a measure becomes a target, it ceases to be a good measure," also applies to scorecards. When an organization treats them at face value without considering their inherent limitations and chooses to chase target numbers blindly, scorecards can do more harm than good.

Finally, a scorecard is always a selection of KPIs. You cannot measure everything. The scorecard narrows the view, and things that are not being measured may get overlooked or even entirely forgotten over time. This "tunnel vision" can be quite risky, because long-term production health may deteriorate unnoticed. When problems eventually sur-

face, the situation is often so bad that it takes tremendous effort and time to get back on track.

Why Are Scorecards Hard to Get Rid Of?

While a scorecard may not capture production health perfectly, it does provide objective facts. Discussing a number is much easier than discussing a sentiment. Scorecards can help reduce conflict and create traction for previously unmanageable production health improvement effort.

Not all teams dislike them. The teams that have great numbers on the scorecard, for whatever reason, may be very proud. Often they have put in hard work to get there. Removing the scorecard without a reasonable replacement may disorient these teams. They might feel that their work is not recognized and that the organization lacks direction. This scenario can undermine morale and jeopardize future attempts at improving production health.

Finally, long-term production health is complex. Most teams do not have the time or capacity to go very deep—they are focused on other priorities, such as dealing with the latest outage or writing code for a feature request. Scorecards make a highly abstract problem tangible. A tool with limits may be better than no tool at all.

Often, acknowledging the limits of scorecards and leveraging what they are good at makes more sense than blindly chasing numbers. They can be useful when used with care.

Case Study: ProdEx—Google's Production Excellence Program

Google SRE's program to assess production excellence, ProdEx, demonstrates a use of scorecards in a way that minimizes their risks. The ProdEx program was created in 2015, and it has conducted over 1,900 reviews since. The tool that Google SRE built for ProdEx aggregates metrics from across production health domains in a central scorecard. These metrics include on-call workload, alerting quality, outages, postmortems, SLOs, data integrity, resource efficiency, bug hygiene, and OKR discipline. Teams supplement this data by referencing their relevant team processes and charter documents. The idea is to provide a comprehensive, high-level overview of an SRE team's health and the state of the production services it is responsible for.

Live reviews via videoconference are held every few quarters with each SRE team using the ProdEx scorecard. Psychological safety is a key element of these reviews. The re-

viewers are two very senior SREs—directors or principal engineers—who are from other parts of the organization and are not part of the leadership chain responsible for the team. This organizational distance reduces the pressure on the team to "show good numbers." The focus is on accountability without shame and coaching instead of judgment.

While ProdEx is highly data-driven, the reviewers are trained to take the team's context into account and to use the review conversation to enrich the picture the data is painting. The scores and action items that are the outcome of the ProdEx reviews are manually defined by the reviewers to ensure a fair assessment.

This combination of data and empathy has helped establish the ProdEx program as a cornerstone of the Google SRE organization's strategic situational awareness, with Ben Treynor Sloss describing it as "one of SRE leadership's most important bits of telemetry about the health of SRE teams." At the same time, ProdEx remains popular with the vast majority of reviewed teams—not bad for what is essentially an audit program.

Effective Scorecard Use

Any organization, regardless of size, requires some governance and change management to effect transformation. These processes vary in terms of formality, consistency, and effectiveness, but they often include some type of health check, scorecard, or maturity model.

Here we discuss some firms that worked with Google Cloud Consulting to establish and grow SRE practices. They provide implementation examples that you might find useful for your own scorecard usage. Each had a desire for some type of scorecard to measure SRE progress, and variations on the scorecard model helped each achieve its goals in a way compatible with its culture and starting point.

Case study: Large financial services firm

One of the primary barriers to wide, efficient SRE adoption in this firm was the existence of several verticals that had been in place since before its employees had computers on their desks. Any violation of these silos' borders was an immediate alarm-raiser, and if governance of SRE transformation was to be done, each vertical wanted to own it for themselves.

In this case, a focus on a *capabilities* model, rather than a *maturity* model, proved to be the answer. Capabilities can be technical (automated rollouts, pervasive tracing), organi-

zational (shared, reallocatable headcount between dev and SRE), operational (clear and consistent postmortem processes), or cultural (consistently blameless culture). These capabilities also lie along a developmental pathway, with some strictly dependent on developing others first. The exact constellation of necessary capabilities varies by organization and even by product, based primarily on reliability targets but also on market specifics, organizational history, and a variety of other factors.

By developing a baseline list of required capabilities to be adopted by all verticals, and then customized capabilities maps for each major product area within those verticals, we were able to incentivize verticals to adopt shared approaches to SRE, while still feeling like they had enough control and individualization to maintain their sovereignty. Scoring could then be done in terms of individual verticals' and products' advancement relative to their specific required capabilities, rather than on an arbitrary, linear, five-level maturity scale. This approach has encouraged individual services to iterate rapidly rather than spend their time on intra- and inter-vertical comparisons and finger-pointing.

Case study: National retail chain

In this case, a national retailer was focused on improving the reliability of its internal logistics applications. Careful and controlled development of SRE practices over nearly a decade had resulted in strong results on the consumer side, but the internal applications were still owned by legacy, infrastructure-focused teams. Much of the ongoing development was handled by a few people tweaking applications written years ago by long-departed contractors.

In this case, the company was not simply looking to apply new SRE approaches to old applications; they also needed to modernize, and in some cases redevelop, the applications for which they wanted to stand up reliability practices. We proposed a tandem model tying application uplift to SRE practice development, creating shared fate and tight integration between the nascent development teams and the SRE organization.

In the model we implemented, each service was compared to a template "model service," in accordance with its criticality and usage level. This determined its current state relative to a baseline set of capabilities. For example, a "tier 1" service might require automated rollbacks and be deployed as a set of containers in the organization's current production-standard Kubernetes environment, while a "tier 3" service might for the moment remain on legacy virtual machines.

A shared backlog was then created for developers and SREs to address jointly. Primary scoring was done on the basis of accomplishment of backlog items relative to the whole,

individualizing the process to the specific services' needs. Rather than SREs being seen as an outside force coming in to fix an application, they were seen as partners, particularly because they arrived with engineering time available for addressing the backlog and driving applications toward the required maturity.

Conclusion

Building a mature SRE function in any organization is a multiyear endeavor. A healthy SRE culture and a working organizational structure are inherently intertwined. Ideally, they support each other.

While any successful SRE team has dedication to and expertise in reliability engineering—in addition to critical aspects of SRE culture, such as blamelessness—deep business understanding and collaboration are also important. No matter where you end up on the centralization spectrum, these tenets are essential to maximizing your SRE function's impact and usefulness.

As you implement these organizational structures, it is particularly important to keep an eye on the maturity and performance of SRE itself, the services it supports, and SRE engagements. This process can have both positive and negative consequences—if you measure the wrong aspects or the right aspects in the wrong way, it can hinder organizational development and alienate the engineers involved. Be conscious and critical about what you measure.

Since the journey is long, keeping track of progress is crucial. A maturity model can help you understand where you are, as well as next steps. SREs love data to support their decision making, so clear success metrics can help guide the journey. However, always be aware that scorecards are only proxies. While they can do a lot of good, they can also derail SRE work when taken too literally as a statement that "team A is better than team B," dividing "winners" and "losers." Keeping psychological safety top of mind as the most important ingredient for a healthy SRE organization will pay dividends on this journey.

1 See Gene Kim et al., *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations* (IT Revolution, 2016).

2 Melvin Conway, "How Do Committees Invent?", *Datamation*, 14, no. 4 (1968): 28–31.

[3] "Shadow SRE" is an analogy to "shadow IT", a common term of art: ad hoc IT functions that spring up in dysfunctional organizations as individual branches of the organization try to solve for their needs in a non-coordinated fashion.

[4] See Chapter 7 of *Seeking SRE*, curated and edited by David N. Blank-Edelman (O'Reilly, 2018).

[5] If your team supports third-party or open source software, your setup will be different. There is still active development, but it happens outside your organization. Establishing communication channels and trust between the product development and SRE teams is equally valuable, but the approach needs to be tailored to the development model.

[6] By "leadership," we mean an individual or group who has influence over an organization and guides it toward a shared goal.

[7] W. Edwards Deming, *The New Economics* (MIT Press, 1993), 35.

[8] Roger Dooley, "A Two-Minute Masterclass from Jeff Bezos - Five Powerful Insights", *Forbes*, December 9, 2024.

Previous chapter
< 5. SRE Hiring, Training, and Career Management

Next chapter
7. SRE Engagement Models >